

DevOps Pipeline Orchestrator

CLI tool and dashboard for managing multi-environment deployments with automated rollback, health checks, and Slack integration.

TypeScript	Node.js	Docker	Redis
------------	---------	--------	-------

VERSION	STATUS	LICENSE	RUNTIME	OWNER	UPDATED
2.4.0	Live	Apache-2.0	Node 20 LTS	Platform Eng	Sep 2026

1. Overview

The DevOps Pipeline Orchestrator is a deployment control plane that promotes a single immutable artifact through an ordered chain of environments — typically `dev` → `staging` → `prod` — under declarative policy. Teams describe the chain once in a `pipeline.yaml` committed beside their service code; the CLI (`pipelinctl`) and the web dashboard are thin clients over the same REST and WebSocket API, so a run started from a terminal is observable in the browser and vice versa.

The design goal is that no deployment requires a human to hold state in their head. Every promotion is gated on machine-checkable health signals, every failed gate triggers an automatic rollback to the last known-good revision, and every state transition is broadcast to Slack with the run ID, the diff, and the actor who initiated it.

2. System architecture



Figure 1 — Request path for a single promotion. Stages are ordered; a gate failure short-circuits to the rollback controller.

Redis is the only stateful dependency in the control plane. It carries three distinct workloads: BullMQ job queues for stage execution, SET NX PX locks that guarantee one active run per (service, environment) pair, and a hash-backed run journal used to rebuild dashboard state after a reconnect. API and worker processes are stateless and scale horizontally behind a standard load balancer; losing any single worker mid-run causes the stage lease to expire and another worker to resume from the last committed checkpoint.

3. Core components

COMPONENT	RESPONSIBILITY	FAILURE BEHAVIOUR
Control API	Auth, run submission, manifest validation, WebSocket fan-out to dashboard clients.	Stateless; drains in-flight requests on SIGTERM.
Scheduler	Expands the manifest into a stage DAG, enforces concurrency locks, enqueues work.	Re-derives the DAG from the journal on restart.
Executor pool	Runs each stage in an ephemeral container with a scoped credential mount.	Lease expiry returns the stage to the queue.
Health gate	Polls HTTP, TCP and custom probes until the quorum and soak window are satisfied.	Timeout is treated as failure, never as success.
Rollback controller	Re-pins the environment to the previous revision and replays its config bundle.	Idempotent; safe to invoke repeatedly.
Notifier	Emits threaded Slack updates keyed by run ID, plus webhook delivery with retry.	Best-effort; never blocks a promotion.

4. Pipeline manifest

The manifest is the unit of review. Schema validation runs in the CLI and again on submission, so an invalid gate is rejected before any container starts.

```
apiVersion: orchestrator/v2
service: checkout-api
artifact: ghcr.io/acme/checkout-api:${GIT_SHA}
environments:
  - name: staging
    strategy: rolling          # rolling | blue-green | canary
    gate:
      probes: [{ type: http, path: /healthz, expect: 200 }]
      quorum: 3/3              # consecutive successes required
      soak: 90s                # hold before promotion is allowed
      on_failure: rollback
  - name: prod
    strategy: canary
    canary: { steps: [5, 25, 50, 100], interval: 5m }
    requires_approval: true    # two-person rule, enforced by API
    gate:
      probes: [{ type: prometheus, query: "rate(http_5xx[2m]) < 0.01" }]
      notify: ["#deploys", "#oncall-checkout"]
```

Listing 1 — A two-environment manifest with a canary rollout and a metric-backed gate.

5. Rollback semantics

- **Trigger.** A gate that fails its quorum, exceeds its soak window, or returns a hard error moves the run to ROLLING_BACK within one probe interval.
- **Target.** The controller resolves the last revision that both deployed cleanly and passed its gate, not merely the previous deploy, so a rollback cannot land on a known-bad build. Artifact tag, environment, and secret references are restored as one bundle.
- **Scope and cutoff.** Database migrations are out of scope and must stay backward-compatible for one release. Bundles are pruned after 24 hours.

6. Performance and limits

DIMENSION	MEASURED	NOTES
Control-plane overhead	0.8 s p50 / 2.1 s p95	Submission to first executor start, excluding image pull.
Concurrent runs	250 per worker pool	Bounded by Redis connection count, not CPU.
Dashboard latency	< 150 ms	WebSocket push from state transition to render.
Rollback time	11 s median	Rolling strategy, 6-replica service.

7. Deployment and operations

- Ships as a Helm chart and a Compose file. The reference topology is three API replicas, four workers, and a Redis instance with AOF persistence enabled.
- Executor credentials are short-lived tokens minted per stage and scoped to the target environment; the orchestrator never stores long-lived cloud keys.
- Prometheus metrics cover queue depth, stage duration, gate outcomes, and rollback counts; queue depth is the recommended paging signal. JSON logs carry the run ID on every line.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
2.5	Manifest-level dependency ordering across services in a shared release train.	Q4 2026
2.6	Pluggable gate providers (Datadog, Honeycomb) behind a stable probe interface.	Q1 2027
3.0	Multi-region control plane with Redis replication and regional executor affinity.	Q2 2027